This is an indication that the optimal value f^* is intimately linked to the optimizer x^*, and while we lose some information by altering our objective, searching for f^* is a good proxy for searching for x^* while being much more computationally tractable.

6.2.2 Implementing entropy search with BoTorch

Having discussed the high-level idea behind MES, we are now ready to implement it using BoTorch and plug it in to our optimization pipeline. The MES policy is implemented by the `qMaxValueEntropy` class in `botorch.acquisition.max_value_entropy_search` as a PyTorch module, similar to most BayesOpt policies we have seen. When initialized, this class takes in two arguments: a GPyTorch GP model and a set of points that will be used as samples in the approximation procedure described in the previous section.

While there are many ways these sample points can be generated, one particular way we have learned from Section 5.3.2 is to use a Sobol sequence, which does a better job covering the targeted space. Overall, the MES policy is implemented as follows:

```
num_candidates = 1000   ❶
sobol = torch.quasirandom.SobolEngine(1, scramble=True)   ❶
candidate_x = sobol.draw(num_candidates)   ❶

candidate_x = 10 * candidate_x - 5   ❷

policy = botorch.acquisition.max_value_entropy_search.qMaxValueEntropy(   ❸
    model, candidate_x   ❸
)   ❸

with torch.no_grad():   ❹
    acquisition_score = policy(xs.unsqueeze(1))   ❹
```

❶ Generate the samples between 0 and 1 using a Sobol sequence.

❷ Rescale the samples to be inside the domain.

❸ Declare the MES policy object.

❹ Compute the acquisition scores.

Here, `num_candidates` is a tunable parameter that sets the number of samples you'd like to use in the MES computation. A larger value would mean an approximation with higher fidelity but it would come at a higher computational cost.

Let's now apply this code to our running problem of optimizing the one-dimensional Forrester function, as implemented in `CH06/01 - BayesOpt loop.ipynb`. We are already familiar with most of this code, so we won't go into the details here.

Figure 6.13 shows the progress made by MES for ten queries, in which the policy quickly finds the global optimum of Forrester after five queries. Interestingly, as optimization progresses, we become more and more certain that looking at other regions in the search space will not result in

©Manning Publications Co. To comment go to liveBook
https://livebook.manning.com/#!/book/bayesian-optimization-in-action/discussion

Licensed to ashwin sampath <ashwin.manning@gmail.com>